

Turning Study Notes into Soundscapes for Better Learning

Sara Hulyalkar

Dhirubhai Ambani International School

Mentor: Mr. Aman Kesarwani

Introduction

This project addresses a critical and overlooked problem in the Indian education system: a dearth of accessible, engaging and personalised learning tools for students balancing academics, long commutes and various other commitments. Rote learning, one of the main learning tools utilised in the Indian education system, has been criticized for numerous reasons. Numerous students tend to struggle with traditional learning methods, they fail to engage students for prolonged periods of time and it doesn't support long-term memory retention. The [Annual Status of Education Report \(ASER\)](#) shows that over 50% of students in Grade 8 cannot solve basic math problems. Meanwhile, **exam stress is taking a tragic toll:** the [NCRB](#) reported over 13,000 student suicides in 2022, with academic pressure as a major contributor. To make content more engaging, especially for auditory learners, neurodiverse students (e.g: ADHD, dyslexia), and young learners, we developed our project Edubeats. Edubeats is an app aimed at enhancing memory retention and focus by converting text-based study notes into corresponding musical numbers, tapping into the cognitive benefits of auditory learning.

Relevance of the Problem in the Real World

In today's fast-paced, screen-saturated learning environment, students face cognitive overload and distractions especially in a city like Mumbai. Traditional text-based study methods often fail to engage learners with diverse needs, particularly auditory learners, students with learning differences (such as ADHD, dyslexia, or processing disorders), and younger students with shorter attention spans.

According to UNESCO and the World Bank's 2023 report on Inclusive Education, over one-third of students worldwide struggle with traditional rote learning techniques, and nearly 20% benefit from multi-sensory approaches. Despite this, mainstream education tools still heavily favor visual and linguistic formats, leaving auditory learners underserved.

At the same time, the rise of digital learning platforms has highlighted both a need and an opportunity for personalization. But most apps that we researched to understand a need for the product focus on gamification, flashcards, or AI tutors while neglecting the unique power of sound and rhythm in cognition and memory formation.

This problem is very relevant and important today because of the limiting attention span where researchers talk about our attention span dropping to 8 seconds. Along with this the rise in screen time has led to other mental health disorders and digital fatigue. Sound based learning offers a more active yet accessible way to let students learn while avoiding burnout.

Despite the rise of AI, few tools allow users to “hear their notes” combining mnemonic reinforcement with creativity. We wanted to address this overlooked space by allowing users to convert plain notes into immersive, thematic soundscapes.

By investigating, designing, and testing a sound-based learning application, this project seeks to fill a critical gap in how we approach study reinforcement. The findings and prototype can influence how educators, app developers, and curriculum designers view auditory learning not as an add-on, but as a core method of content delivery. Ultimately, we aim to create a more inclusive, efficient, and engaging study environment, especially for students who need it the most.

Significance of the Study

The data gathered through user testing, feedback loops, and cognitive impact metrics could serve as the basis for new guidelines in personalized learning, accessibility in edtech, and curriculum augmentation. Moreover, these insights can help educators, school leaders, and app developers make informed decisions about incorporating multi-sensory elements into study tools. Just as sanitation protocols evolve based on surface hygiene data, pedagogical strategies can evolve based on how different senses contribute to learning outcomes. Ultimately, the project promotes a more inclusive, engaging, and neurodiverse-friendly approach to learning, redefining what effective study support can look and sound like in the 21st century.

Real-World Applications and Benefits

The EduBeats project has broad and practical applications across educational, therapeutic, and inclusive learning environments, offering tangible benefits to students, teachers, parents, and the larger edtech ecosystem.

EduBeats provides a creative and effective method for revision by allowing students to convert their notes into immersive audio formats. This helps in reinforcing key concepts using sound memory and rhythm; improving retention and concentration during independent study, and making repetitive revision less monotonous and more enjoyable.

Students with ADHD, dyslexia, auditory processing disorders, or general attention difficulties benefit significantly from auditory cues and rhythm-based learning. EduBeats offers a less text-heavy, more engaging learning experience and customization for different attention

spans or sensory preferences. It is a tool that aligns with Universal Design for Learning (UDL) principles

We also believe Schools can integrate EduBeats in classroom accessibility toolkits and special education support programs creating blended learning models, allowing students to choose how they absorb material.

Despite the [NEP 2020](#) urging personalized, experiential learning, most tools remain one-size-fits-all, making our learning tool increasingly significant for a lot of students in today's world.

Students are no longer passive consumers of notes. EduBeats enables them to **personalize their study environment** through sound themes. Build an **emotional connection** with subjects via storytelling through sound and learn through a **creative, self-directed process** that strengthens motivation

Broader Impact

This project provides a practical and replicable model for integrating sound-based reinforcement into mainstream education and personal learning systems. While EduBeats focuses on converting academic notes into soundscapes, the underlying framework personalized auditory learning can be extended to a wide range of subjects, age groups, and educational contexts.

By demonstrating a clear connection between multi-sensory engagement and improved focus, retention, and motivation, the study offers actionable insights for students, educators, curriculum developers, and education policymakers. It highlights the need to design learning environments that reflect cognitive diversity rather than relying on one-size-fits-all formats.

The methodology, user feedback, and findings from this project can be used to develop inclusive edtech tools tailored for auditory and neurodiverse learners. It can also be used to Inform teacher training workshops on alternative revision techniques. And hopeful Inspire sound-based modules in national digital education platforms (e.g., DIKSHA, CBSE, ICSE e-content)

We envision our project serving as a foundation for future academic research in cognitive psychology, human-computer interaction, or learning science

Main Aim

To assess the effectiveness of sound-based learning as a tool for enhancing focus, engagement, and retention among students, and to develop a functional, student-led app that transforms textual notes into thematic soundscapes.

Objectives- Table 1.1

Objective	Description
Developing a Functional Sound-Mapping Prototype	Build a working app prototype that automatically converts study notes into sound effects using keyword mapping, ambient layering, and user customization features.
Reflecting on Limitations and Future Improvements	Critically evaluate the current build, document challenges, and suggest next steps for scaling, personalization, and data-backed validation in future versions.
Identifying Opportunities for AI Integration	Explore potential areas where AI can be incorporated into EduBeats to enhance personalization, content understanding, voice generation, and adaptive learning over time.

Hypothesis

If a sound-mapping prototype is built that transforms academic notes into immersive audio using AI-powered keyword detection, then it can serve as a practical model for future learning tools that cater to multi-sensory education needs, it will better support auditory learners and students with attention difficulties and provide a more engaging and personalised study experience, all-in-all, making learning more accessible and inclusive.

Methodology

The methodology used was divided into two primary phases: research and design. The initial research was done on the basic science behind music, memory and learning and how they are interrelated and often overlooked by numerous education systems. Further research was

conducted into similar existing platforms, to help us ideate different features that could be incorporated into the app. Finally, the app was built using the coding platform python.

Comparison of Programming Languages for Developing an Educational Song Generation

App- TABLE 1.2

Method	Description	Pros	Cons
Java	Designing the application using Java's object-oriented programming model, considering the potential integration of Java FX or the Android SDK for a mobile-compatible user interface.	-Highly suitable for native Android application development, enabling easy distribution for students -Strong built-in support for file management and user interfaces via Java FX or Android SDK -Stable performance for real-time audio playback	-Complex and time consuming to implement text-to-music logic -Limited existing libraries for creative audio manipulation -Less flexible for quick experimental changes
C++	Development of the application using the C++ coding language to maximise execution efficiency and control over system-level resources. Audio processing could be	- Rapid processing: useful for generating complex audio files or handling large text inputs -Higher degree of control over audio data and memory, allowing	-Programmer is responsible for managing many low-level details that are handled automatically in other programming languages

	implemented using specialist C++ libraries to leverage waveform manipulation and timing precision.	increased precision when generating music -Capability to integrate with professional audio libraries, such as JUCE	-Significantly longer development time -Limited high-level text processing capability -Not ideal for rapid prototyping or iterative design
Python	Constructing the application using python, employing their high level libraries such as gtts for text-to-speech conversion, pydub for audio manipulation and Tkinter for graphical user interface development. Designed for faster prototyping.	-Extensive library support for text-to-speech conversion, audio editing and user interface creation -Ideal for prototyping and testing different lyric-melody combinations quickly -Easy to integrate new, extensive features into the model -Simple syntax enables faster development and debugging	-Slower execution speed may cause delays with larger input files -Less optimised for mobile development: requires additional frameworks such as Kivy for portability -May face audio latency issues compared to compiled programming languages

Why Python and Machine Learning?

Python was selected as the primary programming language for this study due to its practicality, advanced features and alignment with the project's objectives. Python supports vast and comprehensive libraries specifically suited to the project's objectives of text-to-speech generation, audio generation and rapid prototyping, unlike other coding languages like Java and C++. These libraries, coupled with readily available modules like *gTTS* for text-to-speech conversion, *pydub* for audio manipulation and *Tkinter* for graphical user interface development allow for the effective integration of integral elements of the project. Additionally, python's concise syntax and interpreted nature enable faster development and ongoing testing, which is essential for refining features such as lyric formatting, rhythm matching and beat alignment.

As a widely used and well-documented programming language, Python offers a balance of ease of use and powerful functionality, making it the most practical resource, suitable for the scope and constraints of the investigation.

MATERIALS REQUIRED

Table 1.3

CATEGORY	LIST	SOURCING
Hardware and peripherals	● Laptop/desktop computer ● USB Microphone ● Headphones/speakers ● External storage device	Personally owned or borrowed from the school ICT department
Software and Planning Tools	● Python 3.x ● Google Text to Speech Library (gTTs) ● Pydub - audio editing and manipulation ● Tkinter - user interface development	Downloaded from official repositories (open source, or free to use websites)

	● random, re - text processing ● PyCharm - code editor ● Audacity - manual code refinement	
Planning and Documentati on Tools	● Google Docs ● Google Sheets ● Canva	Accessed online via personal google accounts
Human resources	● Mentor ● Peers and test users	Mentor guidance arranged through scheduled meetings and online sessions and peers recruited informally for testing.

PROCEDURE

Step	Explanation
1. Planning and Research	The project began by conducting research on the educational gap in India and some of the primary reasons for that. Further research was conducted into unique study methods that students can use to make learning more engaging and to turn passive moments throughout the day into active study methods, to break the monotony of traditional rote learning. Building on these insights, clear objectives for the app were outlined: to design, develop and test an application that converts traditional study notes into a structured song to aid memory retention. A development plan was created, outlining important features such as text processing, rhythm alignment and audio output. Required software and hardware identified and procured .
2. Initial Development	A prototype was built on python employing key libraries. Basic code modules were developed to process text input, structure text into a song and generate an MP3 output. This process aimed to create a minimal viable product to be tested in its initial stages.

3. Iterative Testing and Refinement	Each component of the application (text processing, audio synthesis, user interface) was tested individually in short developmental cycles. Adjustments were made based on our observations, which included modifications such as rhythm pacing, the inclusion of more backing tracks for lyrics and enhancements in the quality of audio output.
4. User Testing and Data Collection	A group of student volunteers with access to the prototype provided study notes for various subjects. Each set of notes was processed by the application to generate a unique song. Users then listened to their generated songs and provided feedback on the following categories: clarity, engagement, user experience and perceived memorisation benefit. All the data from tests was collected through a Google Form and responses were recorded on Google Sheets for analysis.
5. Data Analysis	Feedback scores were analysed to identify patterns in user preferences, such as the most effective pacing of songs or beat styles. Comparative graphs and tables were created on Canva and Google Sheets to illustrate the relationship between song structure and user satisfaction. Key trends, limitations and improvement opportunities were documented for inclusion in the final report.

STEPS FOR WRITING THE CODE FOR THE APP

1. Installing Required Libraries

- Installed the following libraries required for the project: gTTS, pydub and ffmpeg
- gTTS: Google Text-to-Speech for generating spoken audio from text
- Pydub: for audio manipulation (combining audio, adjusting the volume, manipulation)
- Ffmpeg: required by pydub for audio processing

▶ `!pip install gtts pydub ffmpeg`

```
from gtts import gTTS
from pydub import AudioSegment
import IPython.display as ipd
```

▶ `from gtts import gTTS`
`import IPython.display as ipd`

2. Leveraging basic Text-to-Speech conversion

```
def text_to_speech_colab(text, filename="output.mp3"):
    tts = gTTS(text=text, lang="en")
    tts.save(filename)
    return ipd.Audio(filename) # Plays audio in Colab
```

A basic code to convert text to speech was written such that a text string can be converted into a .mp3 file and it plays directly inside Colab (the software used to make the application)

This feature was tested on the following examples:

```
# Example usage
text = "a squared, b squared, add them both, c squared!"
audio = text_to_speech_colab(text)
audio # Play the audio in Colab
```

```
# Example usage
text = "When prices go up, demand goes down, when prices go down, demand goes up"
audio = text_to_speech_colab(text)
audio # Play the audio in Colab
```

3. Adding Background Music to the Speech

The converted Text-to-Speech (TTS) audio was mixed with background music, using the following code:

```
!pip install gtts pydub ffmpeg

from gtts import gTTS
from pydub import AudioSegment
import IPython.display as ipd

def text_to_speech_with_music(text, speech_filename="speech.mp3", output_filename="final_audio.mp3"):
    # Generate speech from text
    tts = gTTS(text=text, lang="en")
    tts.save(speech_filename)

    # Load the generated speech
    speech = AudioSegment.from_file(speech_filename)

    # Load background music (You need to upload a file)
    bg_music = AudioSegment.from_file("/content/background-royalty-free-music-emotional-piano-305670.mp3") # Replace with your file

    # Adjust background music volume
    bg_music = bg_music - 20 # Lower the volume (-20 dB)

    # Ensure background music is at least as long as speech
    if len(bg_music) < len(speech):
        bg_music = bg_music * (len(speech) // len(bg_music) + 1) # Loop music if it's too short

    # Trim background music to match speech length
    bg_music = bg_music[:len(speech)]

    # Mix speech with background music
    final_audio = speech.overlay(bg_music)

    # Save and return final audio
    final_audio.export(output_filename, format="mp3")
    return ipd.Audio(output_filename) # Play audio in Colab
```

This was then tested using the same example as before:

```
# Example usage
text = "a squared, b squared, add them both, c squared!"
audio = text_to_speech_with_music(text)
audio # Plays the final audio in Colab
```

4. Identifying Areas of Improvement

The two main features required for the application had been created and the basic prototype was complete. Based on this working prototype, areas for improvement and additions to the app were identified:

- The volume of the background music should be soft to ensure the audio is heard
- The background music track should be as long as the audio so that the song is complete. Therefore, the background music track should be trimmed to fit the audio file

These problems were resolved through writing the following code:

```
!pip install gtts pydub ffmpeg

from gtts import gTTS
from pydub import AudioSegment
import IPython.display as ipd

def text_to_speech_with_music(text, speech_filename="speech.mp3", output_filename="final_audio.mp3"):
    # Generate speech from text
    tts = gTTS(text=text, lang="en")
    tts.save(speech_filename)

    # Load the generated speech
    speech = AudioSegment.from_file(speech_filename)

    # Load background music (You need to upload a file)
    bg_music = AudioSegment.from_file("/content/background-royalty-free-music-emotional-piano-305670.mp3") # Replace with your file

    # Adjust background music volume
    bg_music = bg_music - 20 # Lower the volume (-20 dB)

    # Ensure background music is at least as long as speech
    if len(bg_music) < len(speech):
        bg_music = bg_music * (len(speech) // len(bg_music) + 1) # Loop music if it's too short

    # Trim background music to match speech length
    bg_music = bg_music[:len(speech)]

    # Mix speech with background music
    final_audio = speech.overlay(bg_music)

    # Save and return final audio
    final_audio.export(output_filename, format="mp3")
    return ipd.Audio(output_filename) # Play audio in Colab
```

5. Adjusting the tempo of the background music to fit the audio

After the first round of improvements to the prototype, the speed of the audio and background music not matching was a significant issue that needed to be addressed. To solve this, the librosa library on python was installed, to leverage audio analysis and manipulation. Code then had to be written to extract the tempo of the music by detecting beats in the song and estimating the BPM. Further, the computer was given information on how to calculate the adjustment ratio. For example, if the BPM of the music is 100 BPM, but the target BPM is 120 BPM, the ratio will be 1.2. Code to stretch the music over a period of time, or compress it was then input into python. The code for the same is as follows:

```
!pip install gtts pydub ffmpeg librosa numpy soundfile

import librosa
import numpy as np
import soundfile as sf
from gtts import gTTS
from pydub import AudioSegment
import IPython.display as ipd

def adjust_music_tempo(bg_music, target_bpm=120):
    """Automatically adjust background music tempo to match speech speed"""
    y, sr = librosa.load(bg_music, sr=None)
    onset_env = librosa.onset.onset_strength(y=y, sr=sr)

    # Use updated librosa API for tempo detection
    estimated_bpm = librosa.feature.rhythm.tempo(onset_envelope=onset_env, sr=sr)[0]

    # Calculate tempo adjustment factor
    tempo_ratio = target_bpm / estimated_bpm
    new_y = librosa.effects.time_stretch(y, rate=tempo_ratio)

    # Save modified audio using soundfile
    adjusted_path = "adjusted_music.wav"
    sf.write(adjusted_path, new_y, sr)

    return AudioSegment.from_file(adjusted_path)
```

6. Adding Advanced Features into the App

To enhance user experience, the following feedback was received.

- Have the music fade in and fade out for a more enjoyable song
- Adjust the pitch of the speech

These features were included in the app, the code for which is as follows:

```
# Adjust pitch of speech (if needed)
if pitch_shift:
    speech = speech.set_frame_rate(int(speech.frame_rate * (2 ** (pitch_shift / 12.0))))
```

```
# Smooth fade-in & fade-out
speech = speech.fade_in(1000).fade_out(1000)
bg_music = bg_music.fade_in(1500).fade_out(1500)
```

7. Converting Notes Into Structured Lyrics

To improve the quality of the song, code was written to convert the notes input into the software into a structured song, by turning them into lyrics. The following steps were taken:

- The random library was installed into the software to add random rhyming suffixes to the words in the code
- The re library was installed to search, match and replace text in a flexible manner
- Code was written to split the notes into smaller sentences and to clean each sentence created
- Fun, random suffixes were added to the words by leveraging the random feature through the installed library
- Code was written to rejoin the sentences as lyrics

The exact code is as follows:

```

!pip install gtts
!pip install pydub

import random
import re
from gtts import gTTS
from pydub import AudioSegment
import IPython.display as ipd

def format_notes_as_lyrics(notes):
    """
    Converts structured notes into a song-like format by adding rhythm and rhyme.
    """
    lines = notes.split(". ") # Split notes into sentences
    lyrics = []

    for line in lines:
        formatted_line = re.sub(r'\n', ' ', line) # Remove new lines
        formatted_line = formatted_line.strip()

        if formatted_line:
            # Add random simple rhyming suffixes
            if random.random() > 0.5:
                formatted_line += random.choice(["", " oh yeah!", " let's go!", " can you feel it?"])

            lyrics.append(formatted_line)

    return "\n".join(lyrics)

def text_to_speech(text, filename="song.mp3"):
    """Converts text to speech and saves as an MP3 file."""
    tts = gTTS(text=text, lang='en', slow=False)
    tts.save(filename)
    return AudioSegment.from_file(filename)

```

8. Generating Randomised Rhyming Phrases to Enhance Lyrics

To leverage this feature, a small pool of rhyming phrases were defined in Python. These were inserted randomly into the lyrics generated by the previous code to enhance the quality of the

```

def generate_rhyming_phrase():
    """Generates a random rhyming phrase to enhance lyrics."""
    phrases = [
        "Oh yeah, that's the way!",
        "Let's sing it out today!",
        "Feel the rhythm, don't delay!",
        "The melody will light the way!"
    ]
    return random.choice(phrases)

def format_notes_as_lyrics(notes):
    """
    Converts structured notes into a song-like format by adding rhythm, rhyme, and a chorus.
    """
    lines = notes.split(". ") # Split notes into sentences
    lyrics = []

    for i, line in enumerate(lines):
        formatted_line = re.sub(r'\n', ' ', line) # Remove new lines
        formatted_line = formatted_line.strip()

        if formatted_line:
            # Add rhyming phrases and a chorus every few lines
            if i % 3 == 0:
                lyrics.append("(Chorus) " + generate_rhyming_phrase())

            lyrics.append(formatted_line + " " + generate_rhyming_phrase())

    lyrics.append("(Outro) " + generate_rhyming_phrase())

    return "\n".join(lyrics)

```

song.

9. Conversion of files to Speech

This was done to eliminate the need to manually type notes into the app. Code was written to help the app read notes from an existing file and convert that into lyrics to produce a song. New libraries were introduced into the software at this stage:

- pytesseract - extracts text from images so the app can convert images into songs
- pdfplumber - extracts text from pdfs for the software to convert it into a song
- PIL.image - opens image files for pytesseract
- ipywidgets and IPython.display - used for interactive file uploads in Python
- BytesIO - treats uploaded file content as an in-memory file

After introducing these libraries, text extraction was enabled from different types of files, rhyming phrases were generated, the notes were converted into lyrics, sentences were looped and choruses and generated rhymes were added to the song. The code for the same is as follows:

```
import random
import re
import pytesseract
import pdfplumber
from gtts import gTTS
from pydub import AudioSegment
from pydub.playback import play
import IPython.display as ipd
from PIL import Image
import ipywidgets as widgets
from io import BytesIO
from IPython.display import display, clear_output

def extract_text_from_file(uploaded_file):
    """Extracts text from PDF, JPG, PNG, or TXT files."""
    file_name = uploaded_file.name.lower()
    file_content = uploaded_file.content.tobytes()

    if file_name.endswith(".pdf"):
        with pdfplumber.open(BytesIO(file_content)) as pdf:
            return "\n".join([page.extract_text() for page in pdf.pages if page.extract_text()])
    elif file_name.endswith((".jpg", ".png")):
        return pytesseract.image_to_string(Image.open(BytesIO(file_content)))
    elif file_name.endswith(".txt"):
        return file_content.decode("utf-8")
    else:
        raise ValueError("Unsupported file format. Please provide a PDF, JPG, PNG, or TXT file.")

def generate_rhyming_phrase():
    """Generates a random rhyming phrase to enhance lyrics."""
    phrases = [
        "Oh yeah, that's the way!",
        "Let's sing it out today!",
        "Feel the rhythm, don't delay!",
        "The melody will light the way!"
    ]
```

```

# User input: text or file
user_choice = input("Do you want to enter text manually or upload a file? (text/file): ").strip().lower()

if user_choice == "text":
    notes = input("Enter your notes: ")
    display("Generating Lyrics...")

    # Convert notes to song lyrics
    song_lyrics = format_notes_as_lyrics(notes)

    # Convert lyrics to speech
    audio = text_to_speech(song_lyrics)

    # Add background music
    add_background_music(audio)

    # Play the generated song
    display(ipd.Audio("final_song.mp3"))

elif user_choice == "file":
    upload_widget = widgets.FileUpload(accept='.txt,.pdf,.jpg,.png', multiple=False)
    display(upload_widget)
    upload_widget.observe(process_uploaded_file, names='value')
else:
    raise ValueError("Invalid choice. Please enter 'text' or 'file'.")

```

Do you want to enter text manually or upload a file? (text/file): text
Enter your notes: Photosynthesis: The Energy Transformation Process Photosynthesis is a biochemical process in which green plants, algae, and some bacteria
Generating Lyrics...

0:00 / 2:42

```

def format_notes_as_lyrics(notes):
    """
    Converts structured notes into a song-like format by adding rhythm, rhyme, and a chorus.
    """
    lines = notes.split(". ") # Split notes into sentences
    lyrics = []

    for i, line in enumerate(lines):
        formatted_line = re.sub(r'\n', ' ', line) # Remove new lines
        formatted_line = formatted_line.strip()

        if formatted_line:
            # Add rhyming phrases and a chorus every few lines
            if i % 3 == 0:
                lyrics.append("(Chorus) " + generate_rhyming_phrase())

            lyrics.append(formatted_line + " " + generate_rhyming_phrase())

    lyrics.append("(Outro) " + generate_rhyming_phrase())

    return "\n".join(lyrics)

def text_to_speech(text, filename="song.mp3"):
    """Converts text to speech and saves as an MP3 file."""
    tts = gTTS(text=text, lang='en', slow=False)
    tts.save(filename)
    return AudioSegment.from_file(filename)

def add_background_music(voice_audio, music_file="/content/background-royalty-free-music-emotional-piano-305670.mp3", output_file="final_song.mp3"):
    """Mixes the generated speech with background music."""
    background_music = AudioSegment.from_file(music_file)

    # Adjust background music length to match voice audio
    if len(background_music) > len(voice_audio):
        background_music = background_music[:len(voice_audio)]

```

An error message was also programmed if an unsupported file type was input into the application. The code for which is as follows:

```

def extract_text_from_file(uploaded_file):
    """Extracts text from PDF, JPG, PNG, or TXT files."""
    file_name = uploaded_file['metadata']['name'].lower()
    file_content = uploaded_file['content']

    try:
        if file_name.endswith(".pdf"):
            with pdfplumber.open(BytesIO(file_content)) as pdf:
                text = "\n".join([page.extract_text() for page in pdf.pages if page.extract_text()])
        elif file_name.endswith((".jpg", ".png")):
            text = pytesseract.image_to_string(Image.open(BytesIO(file_content)))
        elif file_name.endswith(".txt"):
            text = file_content.decode("utf-8")
        else:
            raise ValueError("❌ Unsupported file format. Please provide a PDF, JPG, PNG, or TXT file.")

        if not text.strip():
            raise ValueError("⚠️ No text found in the file. Please check the content.")

        return text
    except Exception as e:
        raise ValueError(f"❌ Error extracting text: {e}")

```


10. Refining the App to Add Confirmation messages

We received feedback to incorporate confirmation messages, such as “Processing...” “uploaded successfully” and “Click below to listen” into the app to enhance user experience. To leverage the same, the following code was written:

```
def process_uploaded_file(change):
    """Processes the uploaded file and generates the song with confirmation messages."""
    clear_output(wait=True)

    if not upload_widget.value:
        return

    uploaded_file = list(upload_widget.value.values())[0]

    display(f"✅ File '{uploaded_file['metadata']['name']}' uploaded successfully! Processing...")

    try:
        notes = extract_text_from_file(uploaded_file)

        display("🎵 Generating song lyrics...")
        song_lyrics = format_notes_as_lyrics(notes)

        display("🗣️ Converting lyrics to speech...")
        audio = text_to_speech(song_lyrics)

        display("🎧 Mixing background music...")
        add_background_music(audio)

        display("✅ Song creation completed! Click below to listen:")
        display(ipd.Audio("final_song.mp3"))

    except Exception as e:
        display(f"❌ {e}")
```

11. Wrote Code to Process the Lyrics

New libraries were installed at this stage to leverage the following features:

- Nltk - Natural Language Toolkit for text processing
- markovify - generates new text using markov chains
- midutil - allows the creation of .mid files
- pyttsx3 - converts text-to-speech offline

Code was written to preprocess the text, to remove punctuation, leaving only letters, numbers and spaces. This makes the text clean for Markov chain modelling. Improved lyrics were generated using the Markov chain which predicts the next word based on the last two words. Code was also written to generate a random melody, such as a basic C major scale. The code for the same is as follows:

```
# Create MIDI File
def create_midi(melody, filename="melody.mid"):
    midi = MIDIFile(1)
    midi.addTempo(0, 0, 120)
    for i, note in enumerate(melody):
        midi.addNote(0, 0, note, i, 1, 100)
    with open(filename, "wb") as output file:
```

```

# Sample study notes
study_notes = """
The mitochondrion is the powerhouse of the cell.
Photosynthesis converts sunlight into chemical energy.
The Pythagorean theorem states that  $a^2 + b^2 = c^2$ .
"""

# Preprocess text
def preprocess_text(text):
    text = re.sub(r'^a-zA-Z0-9\s', '', text) # Remove special characters
    return text

clean_text = preprocess_text(study_notes)

# Generate Lyrics using Markov Chain
text_model = markovify.Text(clean_text, state_size=2)
lyrics = '\n'.join(filter(None, [text_model.make_sentence() for _ in range(4)]))

# Ensure lyrics are not empty
if not lyrics.strip():
    lyrics = "Learning is fun with music!"

# Generate Melody (Simple Random Notes in C Major Scale)
def generate_melody():
    melody = []
    scale = [60, 62, 64, 65, 67, 69, 71, 72] # C Major scale in MIDI
    for _ in range(8):
        melody.append(random.choice(scale))
    return melody

melody = generate_melody()

```

DETAILED PROJECT TIMELINE

Table 2

TIMELINE OF THE EDUCATIONAL SONG-GENERATION APPLICATION PROJECT			
PHASE	DATES/ WEEKS	ACTIVITIES	ESTIMATED HOURS
Planning & Research	January 6th - 27th (Weeks 1-3)	Conducted extensive research into the topic, outlined key features of the application based on these findings, and finalised the programming language that best incorporated core components of the app.	10 hrs
Initial Prototype	January 27th - February 10th (Weeks 3-5)	Developed the first working version of the application, implemented core functions for text input, lyric structuring and MP3 file generation and conducted initial pilot tests to identify bugs in the app.	15 hrs
Reiterative testing	February 10th - March 31st (Weeks 5-12)	Conducted repeated tests with groups of students over various grades and implemented their feedback into the app. Some features we improved were: making the songs more personalised, being able to adjust the pace of the song etc.	20 hrs
Data Analysis	March 31st - April 7th (Weeks 12-13)	Entered the data from the testing phase into google sheets to analyse it, identified gaps in the app and developed comprehensive strategies to address them	10 hrs
Final Prototype	April 7th - April 28th (Weeks 13-16)	Based on the gaps and improvement strategy identified in the previous phase, the final prototype was built and debugged simultaneously throughout the building process	20 hrs
Report Writing	April 28th - May 12th (Week 16-18)	Compiled all project documentation to draft CREST report, ensuring that technical expectations, evaluation and conclusion aligned with assessment criteria.	10 hrs
Review and Finalisation	May 12th - 26th (Week 18-20)	Reviewed the drafted report with the mentor, identified areas of improvement based on the assessment criteria and implemented the changes before finalising the report.	10 hrs

DATA PRIVACY

Privacy Considerations

Numerous privacy considerations had to be examined while coding the app to ensure its smooth operation.

- **Collection of Data:** the application must only collect necessary information (i.e study notes uploaded by the user) and no personal information should be required to operate the app. Users must be informed explicitly of what data is being collected and for what purpose, prior to the app's usage.
- **Environment and Security:** notes uploaded on the app should be stored in a secure, encrypted environment; communication between the client and the server should be encrypted using the TLS or SSL encryption.
- **Safety:** in case of uploaded text including sensitive or personal material, automated filters and safeguards should prevent inappropriate content from being misused or repurposed. Since the app is designed primarily for educational purposes, it must prohibit sharing or publicising user content without explicit consent.
- **Outcomes and Long-Term Impact:** the app must foster a safe learning environment where students can use creative tools without risking their privacy. By embedding privacy by design and by default the application prioritises ethical educational technology.

Implementation of Safety Considerations:

- To ensure the app only collected the necessary information from the notes input into it, code was written to mask any email addresses or phone-like numbers in the input files. The code for the same is as follows:

```
import re
def sanitize_text(text):
    text = re.sub(r'\b[\w.%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', "[EMAIL REDACTED]", text) # mask emails
    text = re.sub(r'\b\d{10,}\b', "[NUMBER REDACTED]", text) # mask phone-like numbers
    return text
```

- To comply with ethical data storage and memory retention standards, python's tempfile module was introduced into the software to delete uploaded files automatically. The code for this is as follows:

```
import tempfile, os

with tempfile.NamedTemporaryFile(delete=True) as tmp:
    tmp.write(uploaded_file.read())
    tmp.flush()
```

- Code was written to display a clear privacy message on the app interface, seeking user consent before processing uploaded data. The code for this is as follows:

```
import ipywidgets as widgets
consent = widgets.Checkbox(value=False, description="I consent to processing my notes securely")
display(consent)
```

CHALLENGES AND METHODS TO OVERCOME THEM

1. Since study notes can be provided in a variety of different formats, extracting text reliably was a significant obstacle. Characters on documents were sometimes misread and PDFs with complex layouts tended to give poor outcomes. To overcome this, we used the pdfplumber library on python, a library designed specifically for the use of PDF extraction, enabling reliable text extraction from this type of file. Additionally, the pytesseract library was added to the code to extract text from images. The goal was to make the app compatible with certain file types and include specific features to ensure notes get extracted accurately.
2. Generating meaningful lyrics was another obstacle we had to overcome. In some of our prototypes generated output sounded robotic and at times monotonous. Simple string manipulation gave repetitive and boring lines. To solve this, the markov chain was introduced into the software

FUTURE IMPROVEMENTS

Overall, we have been extremely happy with the way the app has turned out, but there are certain features that can be added to improve user experience:

1. Allow users to choose the genre of music they like. While we were able to leverage this to a certain extent in the app, including more genres into the software would help

attract more users to the platform. Additionally, more choice should be given on the voice of the singer (e.g: male/female, accents etc).

2. Incorporating AI into the software to convert the notes into lyrics would ensure smarter lyrics. Natural Language Processing (NLP) models can be added to better summarise notes into singable lines. Rhyme and rhythm detection methods should be added to ensure the lyrics flow smoothly and a keyword emphasis feature should be added so that important keywords are repeated in the song for better memory.
3. Advanced learning features may also be incorporated into the app. For example, keywords can be highlighted on the screen as the song is being sung; quizzes can be added to the app after each song to encourage active recall. A spaced repetition playlist can be added onto the app to allow older topics to be remixed into new songs for more frequent revision.
4. To improve the accessibility and inclusivity of the app, subtitles of each song and transcripts can be made available for generated songs to support students with hearing impairments. For visual learners, the app should include visual diagrams that play on the screen while simultaneously playing the song for improved revision quality.
5. Certain technical upgrades can also be included into the app. AI improvements should be made for better voice quality and more natural singing styles. An offline mode on the app should be developed to support learning on the go.

REFLECTIONS

What Worked Well	What Could've Gone Better
The app successfully converted study notes into speech and produced songs, meeting the project's main aim.	The generated lyrics were sometimes repetitive and lacked a smooth flow; the naturalness of them could be improved by adding more advanced NLP features into the app.
The methodology used of breaking the problem down into stages such as note processing, lyric generation and audio conversion provided a clear structure and ensured steady progress.	User testing was limited to a small group of people so the results might not have reflected the preference of a wider audience. Therefore, some important user feedback might have been missed.

Supervision by a trained mentor allowed more advanced features to be included into the app, improving the quality of the final prototype.	The voice output was often robotic and monotonous - more natural, human-like voices would make the songs more engaging and improve the quality of songs.
Iterative testing allowed small adjustments and quick feedback loops, helping identify problems early on in the development stage.	Integration with a wider range of study tools such as flashcards and quizzes was missing which could have improved learning outcomes.

FINAL OUTCOME

This project resulted in a prototype of a working app that could take study notes, turn them into lyrics and convert the lyrics into audio. The outcome showed that my idea was possible as our test subjects reported learning becoming more fun and interactive. While we did face the obstacle of the voice of the song sounding robotic, the lyrics were converted into speech well and key academic terms were preserved during the lyric generation process which means the app still held value as a learning tool.

I have been learning ballet since the age of three and as I've progressed in my ballet journey, I've found it increasingly difficult to balance my schoolwork and ballet simultaneously because of the number of hours spent passively travelling, spending time during rehearsals and while getting ready. This app helped me a lot to turn these passive moments to active learning opportunities in a way that wasn't monotonous and made learning fun. I would love to share the project more widely, especially with students who prefer auditory learning or those who struggle with learning from textbooks and notes. Ultimately, I want this app to be not only a prototype but a tool that genuinely helps people study in a more creative and enjoyable way.

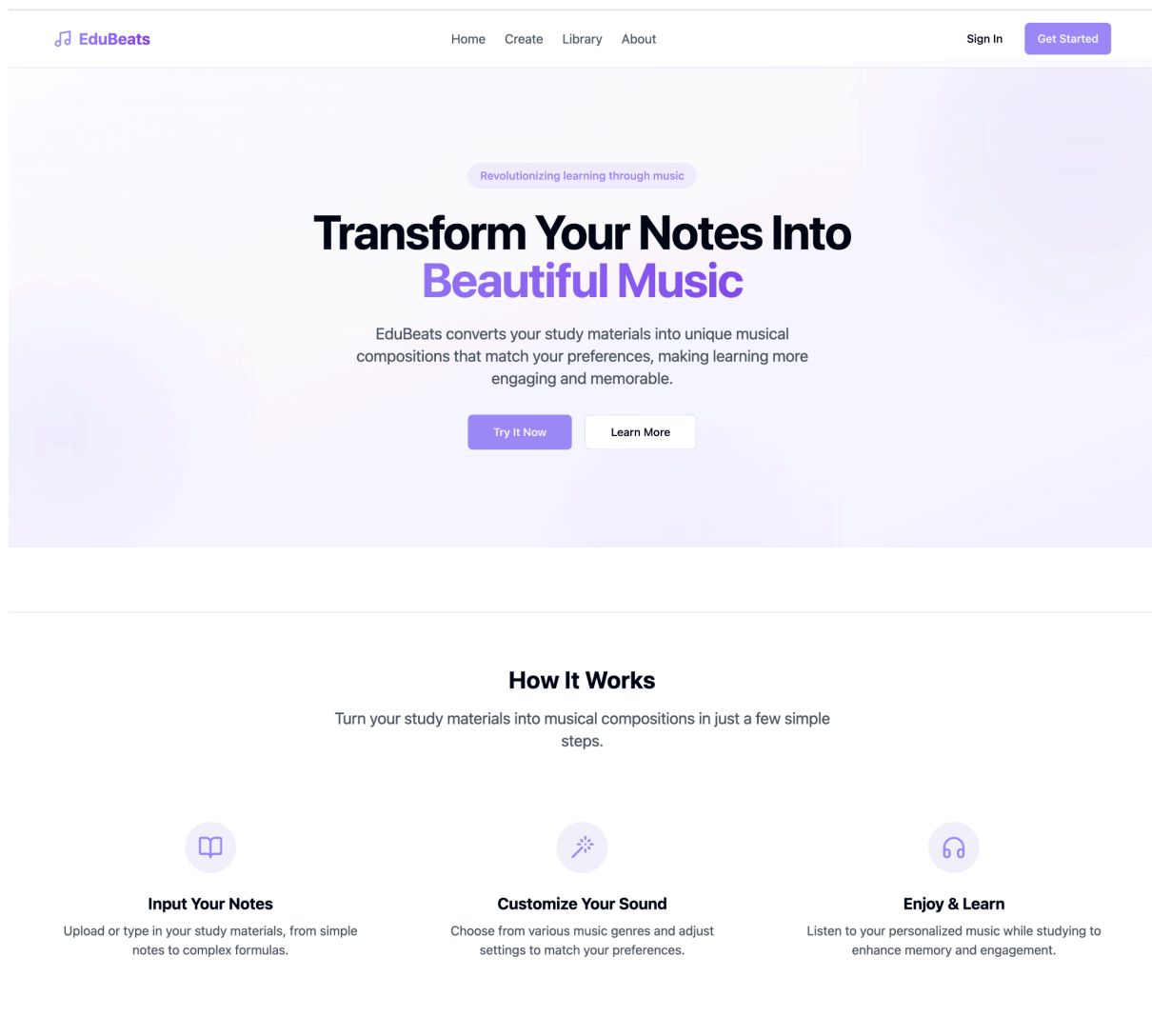
CONCLUSIONS AND IMPLICATIONS ON THE REAL WORLD

- Creative thinking → platforms explored etc
- What we hope this project becomes in the future → dyslexia etc

Documentation

We created a sample interface for our app using an AI tool online called Lovable. Once the prototype of our app is completed, we will transfer it to an interface similar to this.

Home page:



Key Features

EduBeats offers powerful tools to make learning more engaging through music.



Custom Music Generation

Our advanced algorithms convert your notes into unique musical patterns that reflect the content's structure.

- ✓ Multiple genre templates
- ✓ Adjustable complexity and mood
- ✓ Customizable instrument selection



Advanced Text Processing

Our system analyzes your study materials to identify key concepts and structure.

- ✓ Support for plain text notes
- ✓ Keyword and concept extraction
- ✓ Structure-based musical patterns

Create Page:

Home Create Library About

Sign In [Get Started](#)

Create Your Musical Notes

Input your study material and customize the music to match your learning style.

Your Notes

Paste your notes or type them below. The more detailed your notes, the more unique your music will be.

Enter your study notes here...

0 characters • 0 words

Upload File

Transform to Music

Music Style

Customize how your notes will sound as music

Genre

Lo-Fi

Electronic

Classical

Hip-Hop

Relaxed beats with a calm atmosphere, perfect for focused study sessions.

Tempo

85 BPM

Slow

Fast

Complexity

50%

Simple

Complex

Mood

Calm

Calm

Energetic

Music Preview

After processing your notes, your music will appear here.

0:00

3:00

Library page:

 Home Create Library About Sign In [Get Started](#)

Your Beat Library

[All](#) [Lo-Fi](#) [Classical](#) [Electronic](#) [Hip-Hop](#)  



Biology Chapter 5
Lo-Fi • 3:24
[biology](#) [science](#) [cellular](#)
📅 Apr 12, 2023



History World War II
Classical • 4:12
[history](#) [war](#) [europe](#)
📅 Apr 10, 2023



Math Formulas
Electronic • 2:37
[math](#) [algebra](#) [formulas](#)
📅 Apr 5, 2023



Chemistry Notes
Hip-Hop • 3:58
[chemistry](#) [science](#) [elements](#)
📅 Apr 2, 2023



English Literature
Lo-Fi • 3:41
[english](#) [literature](#) [poetry](#)
📅 Mar 28, 2023



Physics Chapter 3
Electronic • 2:52
[physics](#) [science](#) [mechanics](#)
📅 Mar 25, 2023

About the App page:

 Home Create Library About Sign In [Get Started](#)

About EduBeats

EduBeats is an innovative application that transforms educational notes into music, making learning more engaging and memorable through audio association.

Our Mission

At EduBeats, we believe that learning should be engaging, personalized, and effective. Our mission is to revolutionize the way students interact with educational content by leveraging the power of music and sound to enhance memory retention and learning outcomes.

By transforming educational notes into melodic patterns that reflect the structure and importance of the content, we create a multi-sensory learning experience that helps information stick in your memory while making studying more enjoyable.

The Science Behind EduBeats



Neuroscience of Music

Research shows that music activates multiple areas of the brain simultaneously, creating stronger neural connections and improving memory recall.



Musical Memory

Music is processed by the same cognitive systems that handle temporal and sequential learning, making it a powerful tool for encoding structured information.



Multi-Sensory Learning

Engaging multiple senses during study increases attention, enhances encoding of information, and improves overall retention compared to single-modality learning.

How EduBeats Helps Students



Enhanced Retention

By associating study material with unique musical patterns, students can recall information more effectively during tests and examinations.



More Engaging Study Sessions

Turn boring study materials into an enjoyable audio experience that keeps students motivated and focused for longer periods.



Personalized Learning

Customize the music to match your learning style, preferences, and the specific subject matter for a tailored educational experience.

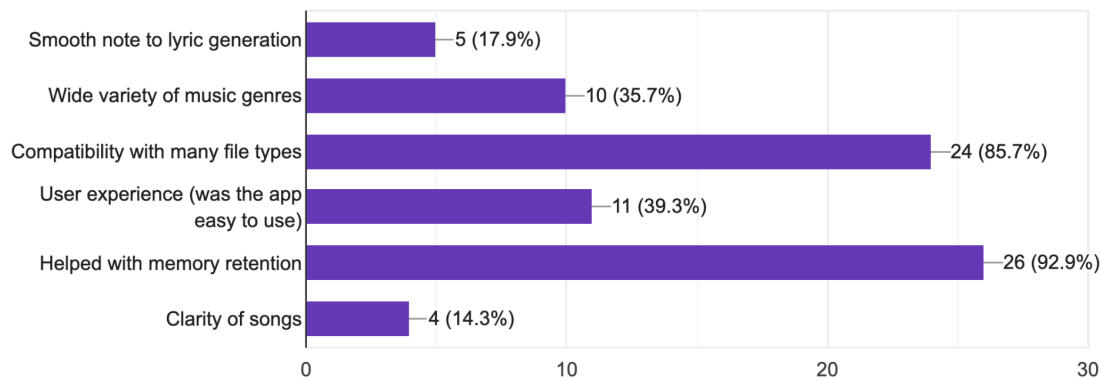
During the ideation stage of our project, we compiled the following documents to help check the feasibility of our app and make the approach to the project clearer. The links are as follows:

- **IdeationDocument:** [Ideation Document](#)
- **Technology required for the app:** [Tech Planner](#)

As mentioned previously, we conducted iterative testing and made improvements to the prototype throughout the development stage. The data we gathered is shown below:

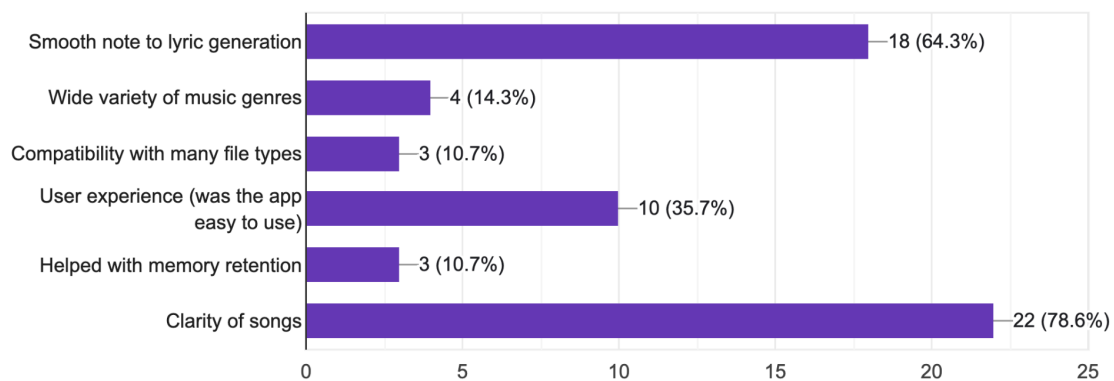
What did you like about our app?

28 responses



What could have been better in the app

28 responses



What features do you recommend we add to our app to improve user experience

19 responses

Make lyrics fade in and fade out to make it more musical

Lyrics on screen

Words weren't very clear

include more potential music genres

More rhyming

include messages on the user interface to make it easier for me to know what to do to get my song

make the lyrics flow smoothly and sound more natural

Make the app accessible in different languages

the tempo didnt match the song which made it tough to follow

What features do you recommend we add to our app to improve user experience

19 responses

add features like quizzes to the app to help promote active recall

make the lyrics available for users to download and refer to once the song has been generated

More personalisation

a feature to highlight key terms would make studying increasingly effective

Make the voice less robotic

include more genres of music into the app to chose from

Make it sound more natural

make the song sound more natural

include pop ups on the app which direct the users on how to navigate the app

CITATIONS AND SOURCES USED

1. Alis. "Dive into Average Human Attention Span Statistics & Facts - Alis Behavioral Health." *Alis Behavioral Health*, 7 Oct. 2024, www.alisbh.com/blog/average-human-attention-span-statistics-and-facts/.
2. Kawata, Natasha Y. S., et al. "Auditory Cognitive Training Improves Brain Plasticity in Healthy Older Adults: Evidence from a Randomized Controlled Trial." *Frontiers in Aging Neuroscience*, vol. 14, Mar. 2022, <https://doi.org/10.3389/fnagi.2022.826672>.
3. rishabh. "Annual Status of Education Report (ASER): Key Education Trends for UPSC 2025 Analysis." *Sleepy Classes IAS*, 11 Jan. 2025, sleepyclasses.com/annual-status-of-education-report-aser/.
4. "Rote Learning versus Critical Thinking in Indian Education." *Pureprofessional.in*, 22 Oct. 2019, pureprofessional.in/public/blogs/506/rote-learning-versus-critical-thinking-in-indian-education. Accessed 5 Sept. 2025.
5. TNN. "Student Suicides Dip Slightly but Mental Health Crisis Deepens, Shows NCRB Data." *The Times of India*, The Times Of India, 21 July 2025, timesofindia.indiatimes.com/india/student-suicides-dip-slightly-but-mental-health-crisis-deepens-shows-ncrb-data/articleshow/122816976.cms. Accessed 5 Sept. 2025.
6. Windsor, Kate. "Auditory Learning Style Explained: Unleashing the Power... | Listening." *Listening.com*, 6 Dec. 2024, www.listening.com/blog/auditory-learning-style.